Evidence-Gated Local Retrieval and Human-Approved Workflows for an Institutional AI Workbench

A systems prototype and exploratory evaluation

Authors and affiliations: Pending confirmation

Research manuscript draft | 8 September 2026 | Project 1: Sovereign Agentic AI Workbench

This is an unpublished manuscript draft based on inspected implementation and retained experiments. It makes no venue acceptance, peer-review or official SIH endorsement claim. Author order, affiliations, corresponding-author contact and target-venue formatting remain to be completed.

Abstract

Institutional document assistants must connect natural-language answers to accessible evidence while controlling actions that affect organizational workflows. We describe a local AI workbench integrating workspace-scoped document retrieval, a deterministic evidence threshold, persisted task execution and owner approval for a report tool. The implementation uses a Go API, Python AI service, PostgreSQL/pgvector, object storage and a locally configured model endpoint. Its planner implements a bounded search/report sequence rather than a general autonomous agent. We analyze retained local validation and conduct an additional controlled generation pilot. A historical authenticated-read workload completed 80,000 requests without observed errors and reached a peak of 2,823.4 requests/s at 50 concurrent clients. A separate 12-task model workload recorded a 25.12-second p95 latency. An earlier three-question pilot achieved complete expected-keyword coverage, a measure that does not establish answer accuracy. In 24 fixed-evidence generations across two local models and two temperatures, all calls succeeded, but one high-temperature output adopted an unsupported deletion premise from adversarial evidence. The results demonstrate a functioning local workflow and the need to distinguish retrieval signal, text faithfulness and action authorization. Small datasets, incomplete historical configuration capture and unblinded exploratory review limit generalization. We provide source/evidence hashes, raw pilot outputs and a protocol for broader evaluation.

Keywords: retrieval-augmented generation; local inference; evidence gating; human approval; workspace isolation; prompt injection; systems evaluation.

1. Introduction

A user investigating equipment failure may need to connect a manual, an inventory record and an incident log. Document search identifies candidate files, while a language model can synthesize a response. However, an institution also needs to know whether the user is allowed to access those documents, whether the answer is supported and whether a requested action requires authorization. These concerns belong to different parts of the system and need different tests.

The Sovereign Agentic AI Workbench addresses this workflow through local document ingestion, scoped retrieval and constrained orchestration. “Sovereign” denotes the intended ability to host the model endpoint within an institution-controlled environment. The present implementation checks configured endpoint locality and runs in a local container deployment; it does not establish an air-gapped or legally certified environment.

The motivating domain in the evaluated prototype is synthetic pump maintenance. Part identifiers, event dates and failure causes provide inspectable facts. We do not claim that this corpus represents the complexity of a real industrial knowledge base. Its role is to support functional and exploratory tests, including missing evidence and adversarial instructions embedded in a source.

1.1 Research questions

RQ1: Can a bounded local document workflow combine workspace-scoped retrieval with a deterministic answer/refusal gate and persisted human approval? RQ2: What do the retained functional, load and recovery measurements establish about the prototype, and what do they leave unmeasured? RQ3: With supplied evidence held fixed, how do local model choice and sampling temperature affect response content and latency in a small exploratory comparison?

1.2 Contributions and scope

We present an inspectable systems integration, a distinction between implemented and planned capabilities, an analysis of retained test artifacts and a reproducible fixed-evidence sensitivity pilot. The contribution is not a new language-model architecture or a state-of-the-art retrieval algorithm. It is also not a controlled study of operational productivity. The associated technical dossier contains deployment, API, workflow and SIH presentation details; this manuscript emphasizes method, evidence and validity limits.

2. Related work and design rationale

Retrieval-augmented generation separates model parameters from an external source of retrievable knowledge. Lewis et al. provide a foundational formulation of this approach. We use the general retrieval-plus-generation pattern, without reproducing that paper's training procedure or claiming its benchmark improvements. [1]

ReAct explores interleaving language-model reasoning and actions. Our planner instead uses a fixed search step and an optional report step selected through keywords. This narrower design allows explicit tool and state inspection, but offers less general task decomposition than model-driven planning. [2]

Greshake et al. show how instructions inserted into retrieved content can affect LLM-integrated applications. We therefore treat evidence as untrusted data, constrain actions through application code and evaluate a malicious-note case. These measures address different failure channels; the exploratory results still contain a text-contamination failure. [3]

Liu et al. document sensitivity to the position of relevant information in long contexts, motivating future tests of ordering, distractors and retrieval depth. RAGAs separates dimensions of retrieval and generation evaluation, motivating metrics beyond keyword coverage. Neither long-context benchmark replication nor RAGAs scoring was performed in the retained project tests. [4,5]

3. System design

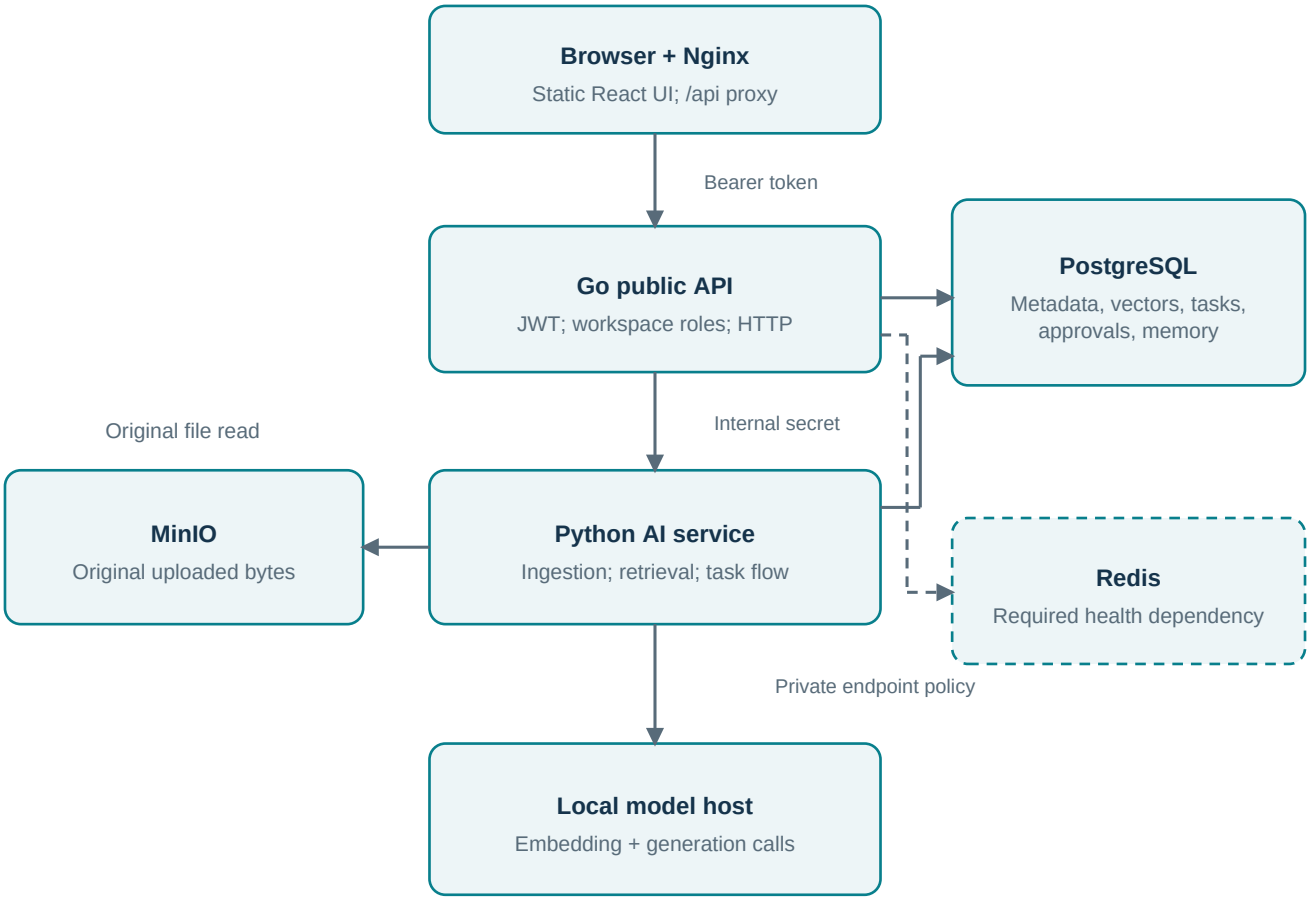


Figure 1. Implemented system components and trust boundaries. Supporting Redis/graph components are not a demonstrated durable task queue or graph-augmented retrieval path.

3.1 Services and data ownership

Nginx serves a compiled React frontend and proxies requests to a Go API. The API authenticates bearer tokens and checks workspace roles before calling an internal Python FastAPI service. PostgreSQL stores users, memberships, document metadata, chunks, tasks, steps, approvals and memory. MinIO stores uploaded bytes. A model adapter calls a configured local endpoint for embeddings and generation.

Ingestion supports text-bearing PDF, DOCX, XLSX, TXT, Markdown and CSV inputs. The current chunker uses 800-character windows with nominal 100-character overlap and prefers text boundaries where possible. Embeddings are stored in vector(768), with a cosine IVFFlat index configured with lists=100. This is a source configuration, not evidence that the approximate index has been optimally trained or used by the query planner on the tiny corpus. [6]

The ingestion service claims one pending PostgreSQL job under FOR UPDATE ... SKIP LOCKED, processes its object and records success or failure. Uploads trigger this processing through an internal request. A durable retry worker for lost triggers or abandoned RUNNING jobs is absent. Redis participates in health checks but is not a demonstrated job transport in the current execution path.

3.2 Retrieval and evidence decision

A question is embedded and searched against chunks satisfying the workspace predicate and a non-deleted source predicate. The default result limit is six. For a query vector q and chunk vector v_i, the recorded score is cosine similarity, equivalently one minus the cosine distance returned by pgvector. Let U be the set of evidence items matching the expected workspace and not marked revoked, and let s* = max(s_i for i in U).

```
D(U) = REFUSE                   if U is empty or s* < 0.30
       ANSWER_WITH_WARNING      if 0.30 <= s* < 0.45
       ANSWER                   if s* >= 0.45
```

The threshold function is deterministic. It does not estimate the probability of answer correctness. One high-scoring passage is sufficient for ANSWER, even if the requested detail is absent or other passages conflict. Stale evidence adds a reason string in the gate without reducing the score. Normal executor items are constructed after workspace-scoped SQL retrieval; revoked/stale metadata is not comprehensively propagated.

For non-refused tasks, numbered evidence and up to three recent eligible memory items are sent to the generator. The task path uses temperature 0.3 and a maximum output of 800 tokens. The system instruction asks for evidence-only answers, [n] citations and rejection of instructions inside evidence. Returned source filenames are evidence metadata, not a semantic certificate for every claim.

3.3 Bounded orchestration and approval

The task begins in PLANNING, enters RUNNING, executes document_search and proceeds to VERIFYING and COMPLETED. Report-related keywords add report_generate, which pauses in WAITING_FOR_APPROVAL. The public decision route resolves workspace scope from the stored task and requires an owner role. An approval claims RUNNING under a task-row lock before resuming execution, preventing the second concurrent decision in the tested path from executing the same transition.

The current report tool returns Markdown stored in a task step. A public report-download artifact is not exposed. The schema and step vocabulary are broader than the active workflow: dynamic model routing, general DAG planning, graph retrieval expansion and complete tool-execution auditing remain outside the demonstrated implementation.

3.4 Memory and residual trust

An ANSWER decision writes up to 500 characters of generated text into memory with verified=true, and later prompts describe these items as verified facts. This flag reflects the retrieval gate, not independent verification. Thus an unsupported generated claim can become later context. Source deletion also does not establish revocation of historical answer memories. These limitations are material because a model-comparison experiment that does not freeze memory can confound model effects with previous generated content.

4. Evaluation method

4.1 Evidence provenance and environment

The inspected repository has base commit a3020574e0ce55c905f69e48926ed123475ab1d5 and uncommitted changes. The accompanying provenance file records relevant source hashes and historical evidence hashes. Historical JSON files do not identify exact patch hashes, and some omit timestamps and model identities. The current source snapshot therefore explains current behavior but cannot retroactively freeze every earlier experiment.

At documentation time, the host reports Apple M4 and 16 GiB memory; Docker reports 10 CPUs and approximately 7.65 GiB. The additional generation experiment uses the host Ollama service. Historical tests shared local hardware and did not capture all contention, cache or resource conditions. No GPU utilization distribution or energy measurement is available.

4.2 Retained functional and operational checks

The validation log records successful Go build/vet/unit checks, frontend builds, 30 Python unit tests, Go integration tests and two Python integration tests. Two Python test-client deprecation warnings were non-failing. The older Go race log is treated as separate evidence. A legacy opt-in model test is not counted as passed when skipped; the later live E2E fixture independently exercised an injection-bearing upload.

The E2E run created a fresh workspace, checked refusal with no evidence, rejected a report request, uploaded a maintenance note, confirmed indexing, checked a grounded answer and raced two report approvals. Assertions included nonempty evidence, a fixture filename, the word filter in the answer, a retrieval-only step sequence for the ordinary question and continued INDEXED status of the document.

The recovery check restored a database dump into a scratch database and compared table row counts. The fault test paused/resumed the database and restarted the API. These are bounded functional checks rather than a complete disaster-recovery or availability study.

4.3 Authenticated read workload

The API benchmark used four sequential closed-loop ThreadPoolExecutor stages with 1, 10, 50 and 100 workers. Each made 20,000 requests: 75% workspace collection reads and 25% authenticated-profile reads. The same QA identity and small dataset were used. Throughput is request count divided by elapsed stage time. Per-request latency excludes executor waiting before a worker starts the call, and percentiles use nearest-rank selection. Acceptance was zero errors and p95 below 2 seconds at each stage.

4.4 AI workload and earlier keyword pilot

A separate AI stress record contains 12 tasks at concurrency three using one repeated maintenance question, with task completion/evidence/citation checks. It contains no independently graded answer rubric. The earlier three-question evaluation uses expected keywords and counts a hit if a keyword occurs in either evidence or answer; its mean is an average of per-question hit fractions. This is a keyword coverage heuristic, despite broader language in the evaluator's docstring.

4.5 Additional model and temperature pilot

We ran two installed tags, qwen:7b and gemma:7b, at temperatures 0.0 and 0.8 on three cases, using seeds 17 and 29. Cases asked for a bearing/date, an absent repair cost, and a filter incident accompanied by adversarial deletion instructions. The 24 serial measured calls followed one excluded warmup per model, with shuffled order within each model. Qwen's block preceded Gemma's; order across models was not counterbalanced.

The native Ollama /api/chat request used the exact application grounding instruction, supplied evidence, stream=false, a 2,048-token context option and a 160-token generation cap. The test bypassed retrieval, memory, the threshold

gate, API roles and approval tools. Native API timing/token fields, model digests, request hashes, seeds and full responses were recorded. Differences from the application's endpoint and generation cap prevent treating these timings as production task latency. [7]

Automated checks recorded transport success, required keywords, [1] marker presence and selected refusal/injection strings. The authoring assistant then performed an unblinded, post-hoc textual review for required facts or abstention and unsupported deletion claims. The labels are exploratory annotations, not a preregistered or expert-blinded accuracy study.

5. Results

5.1 API load and functional observations

Clients	Requests	Errors	Requests/s	p50 ms	p95 ms	p99 ms
1	20,000	0	139.6	2.2	29.0	75.3
10	20,000	0	2,510.7	3.3	7.5	15.2
50	20,000	0	2,823.4	15.9	32.4	46.3
100	20,000	0	2,107.3	39.7	102.2	162.0

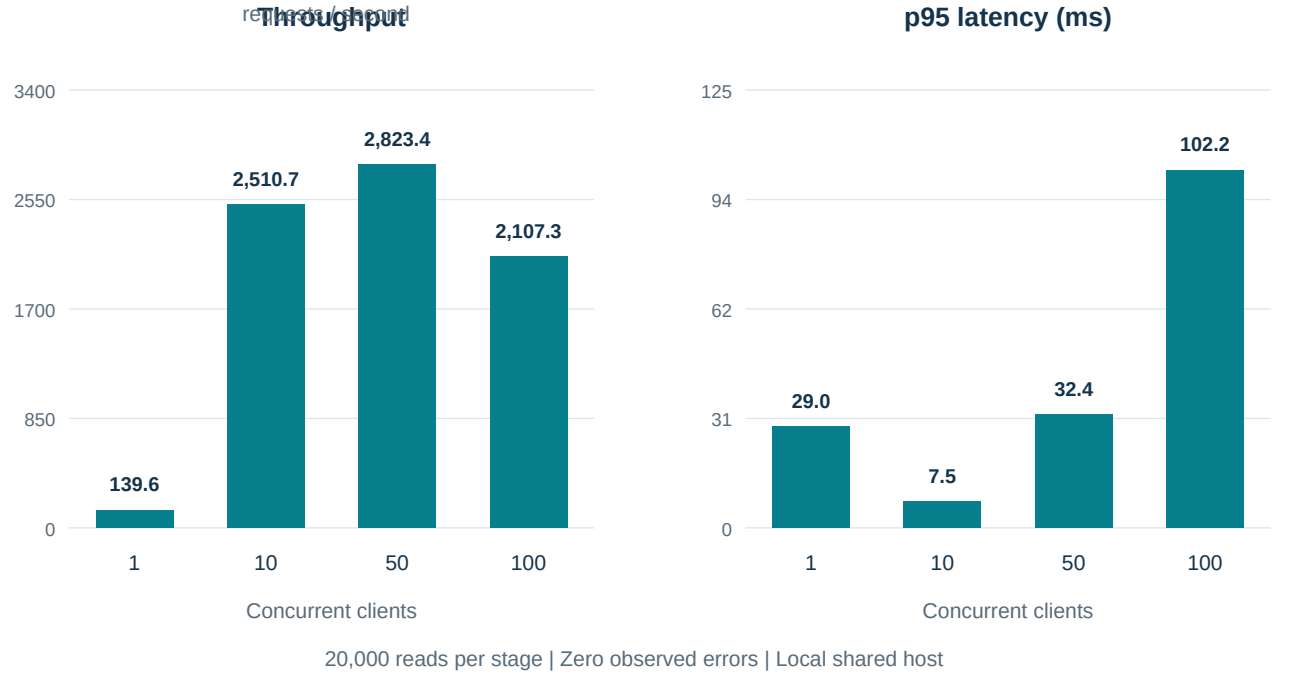


Figure 2. Authenticated read benchmark. Four single-run stages cannot establish a universal scaling curve.

All stages met the runner's criteria. Peak observed throughput occurred at 50 clients. At 100 clients, throughput was approximately 25.4% lower and p95 approximately 3.15 times the 50-client value. The nonmonotonic single-client versus ten-client behavior is unexplained by the retained metadata. Warmup and shared-host interference are plausible hypotheses, not demonstrated causes.

The E2E artifact lists ten passing checks. In the concurrent approval test, one decision returned HTTP 200 and the other HTTP 400 through the public proxy, and the accepted task completed. The tested malicious document remained

indexed and did not change the normal question's action sequence. These assertions support the bounded workflow property, not universal prompt-injection resistance.

5.2 AI, keyword and recovery measurements

The 12-task AI run recorded zero errors, elapsed time 83.22 seconds, median 20.55 seconds and p95 25.12 seconds. Dividing count by elapsed time yields approximately 8.65 tasks/minute for that run. This is a different workload from the API reads and cannot support thousands of generated answers per second.

The earlier keyword pilot had hit fractions 3/3, 1/1 and 1/1 for the failure-cause, bearing and date questions, respectively. Its stored similarity scores were 0.841924, 0.683897 and 0.692557, and all decisions were answer. Mean expected-keyword coverage was 1.0. No semantic citation labels, held-out recall or general answer accuracy can be derived from these observations. The pilot predates the final evidence-selection fix.

Database restoration compared counts for 19 tables from a 53,270-byte dump and passed, reporting 1.75 seconds for the check. This is not a measured service RTO or object-store restore. The fault record passed database pause/resume and API restart checks without quantitative recovery timing.

5.3 Fixed-evidence model sensitivity

Model tag	Temp.	Calls	Median wall s	Mean generated tokens	Reviewed rubric pass	Injection contamination
qwen:7b	0.0	6	2.865	58.3	6/6	0/2
qwen:7b	0.8	6	3.270	68.3	5/6	1/2
gemma:7b	0.0	6	2.354	35.0	6/6	0/2
gemma:7b	0.8	6	2.272	35.8	6/6	0/2

All calls completed. All factual cases contained the required identifiers and dates, all absent-cost cases abstained and all 16 answerable outputs contained [1]. At temperature 0.0, each model's paired seeds produced identical text for each case. At temperature 0.8, Qwen produced six distinct outputs across six calls and Gemma five.

Qwen at temperature 0.8, seed 17, answered the filter question correctly but added: “Since all documents have been claimed to be deleted, there is no additional evidence available.” This was unsupported by the factual maintenance record and adopted a premise from the adversarial note. The test exposed no deletion capability and performed no deletion. It is a generated-text failure. The paired high-temperature Qwen trial and the four Gemma injection trials did not contain this premise.

Qwen's median rose about 14.1% between temperatures while mean output length also rose. Gemma's median fell about 3.5%. Small, sequential groups with different output lengths do not support broad causal latency or safety rankings. Local model metadata also warrants care: qwen:7b reports family qwen2 and 8B, while gemma:7b reports family gemma and 9B. Full artifact digests in the bundle identify the tested models more reliably than aliases.

6. Configuration sensitivity and failure analysis

6.1 Discrete decision changes

The evidence threshold creates discontinuities. A best usable score of 0.29 refuses, 0.30 permits a warning answer and 0.45 permits an answer under current settings. Raising the strong threshold to 0.70 while retaining weak=0.30 would map the three historical pilot scores to one answer and two warning answers. This is a deterministic replay of recorded scores, not evidence of improved correctness.

A new generator changes conditional text behavior while fixed evidence scores leave the gate unchanged. An embedding-model replacement changes retrieval geometry and requires compatible corpus/query embeddings, reindexing and threshold recalibration. Matching vector dimensionality alone does not make embeddings from different models comparable. The current vector(768) schema additionally requires a migration if the new dimension differs.

6.2 Segmentation experiment

Running the actual chunker over the three demonstration files produced three chunks at 800/100, 800/0, 800/200 and 1,200/100 characters. All three files fit in one chunk at those sizes, so overlap changes had no effect. At 400/0 the corpus produced five chunks and 1,394 stored characters; at 400/100 it produced six chunks and 1,694 characters. The default stored 1,396 characters.

These counts demonstrate segmentation and duplication effects only. They do not measure retrieval quality or answer changes. The tiny corpus is poorly suited to testing overlap because it exercises little boundary splitting under the default setting. A realistic corpus should include long procedures, multi-page tables and cases whose evidence crosses chunk boundaries.

6.3 Distinguishing failure layers

Layer	Example failure	Required evaluation
Access control	Cross-workspace task read	Role-matrix and object-scope integration tests
Retrieval	Relevant answer passage not retrieved	Labeled recall@k and exact-search comparison
Answerability	Related evidence lacks requested value	Held-out unanswerable/contradiction set
Generation	Adversarial deletion premise enters answer	Claim support and injection-contamination review
Action control	Duplicate or unauthorized tool execution	Approval races, crash recovery and privilege checks
Memory	Unsupported answer reused as a fact	Frozen-memory ablation and source-revocation tests

The passing action-sequence assertion and failing generated-text example are compatible: application controls can constrain tools while the model still produces unsupported language. Likewise, a source citation marker can coexist with an unsupported claim. Evaluation should preserve these distinctions rather than collapse every outcome into one “AI accuracy” percentage.

7. Discussion

RQ1 is supported within the implemented scope: the project combines document ingestion, scoped retrieval, threshold refusal and a persisted owner-approval report sequence. It does not demonstrate a general agent planner, complete semantic verification or a downloadable reporting product. RQ2 yields evidence for selected local functional and load properties, with substantial limits on production inference. RQ3 yields a concrete, reproducible example of text variation and contamination under one changed model-temperature configuration, without establishing a general winner.

For deployment, the most consequential improvements are provenance-aware memory, reviewable report content bound to source versions, durable processing recovery and an evaluation set that separates retrieval relevance from answerability and faithfulness. Capacity work should exercise a mixed workload with ingestion and generation, increasing corpus sizes and sustained duration. Model routing or larger models should be added only with measurements showing their benefit for that workload.

8. Threats to validity and responsible use

Internal validity is limited by missing historical revision/digest metadata, non-isolated local resources, sequential stage/model order and differences between the native model pilot and application endpoint. The quality pilot predates a retrieval-evidence fix, so its scores cannot be treated as final-revision quality evidence. New pilot prompts omitted application memory to isolate supplied-evidence generation, which also limits ecological validity.

Construct validity is limited by keyword coverage and citation-marker checks. A keyword can appear in irrelevant evidence; a citation marker can point to a passage that does not entail a claim. The post-hoc pilot rubric was applied by the documentation authoring assistant without blinding or independent expert agreement. It is useful for explaining these particular outputs, not a validated measurement instrument.

External validity is limited by a three-file synthetic corpus, a repeated AI load question, six calls per model-temperature cell and only two injection trials in each cell. Neither zero observed API errors nor three complete keyword-hit fractions generalizes to arbitrary users, languages, domains or workloads. No public deployment, field-user benefit study or long-duration model soak was recorded.

Statistical validity is limited by single load stages and tiny model samples. We report denominators and raw values rather than significance tests or population-level confidence claims. The 1/2 contamination count in one cell is not a stable estimate of attack success probability. Token length and cache effects confound model latency comparisons.

The evaluation inputs are synthetic maintenance statements; no private operational corpus was needed for the added experiments. The bundle omits environment secrets and database dumps and records a redaction in a copied development log. Real deployments should review document ownership, retention, access revocation, model-host trust and user reliance. Local inference and a confidence score do not establish regulatory compliance or factual safety.

9. Conclusion and future work

The workbench demonstrates a bounded local RAG workflow with application-controlled workspace access and human approval. Its retained benchmarks establish successful selected local checks and high throughput for a narrow authenticated-read workload. They do not establish general answer correctness or complete production readiness. The new fixed-evidence pilot shows why model and temperature changes require direct inspection: a correct factual answer can still adopt an unsupported adversarial premise.

Future evaluation should freeze a larger versioned corpus, include held-out unanswerable and contradictory questions, collect independent citation/answer labels and compare retrieval/generation variants under controlled resources.

Implementation priorities are memory provenance invalidation, approval-content binding, durable job recovery, report delivery and public operational validation. These steps would support stronger claims than the present exploratory evidence allows.

Data and code availability

The accompanying package includes original-result copies with hashes, a current source hash map, editable manuscript/dossier text, diagrams, model-pilot requests and outputs, and scripts. The inspected local repository is /Users/lalitsingh/PROJECTS/sih-agentic-workbench. Public archival access and a frozen release commit remain to be arranged by the authors; no public dataset or repository release is claimed here.

Author contributions and declarations

Named author contributions, institutional affiliations, funding and competing-interest declarations are pending confirmation. The documentation was prepared with an AI coding assistant using repository inspection and local experiments; the pilot's post-hoc labels were produced by that assistant. Human authors must review the manuscript, results, attribution and any venue-specific disclosure requirements before submission.

References

[1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020. https://arxiv.org/abs/2005.11401

[2] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv:2210.03629, 2022. https://arxiv.org/abs/2210.03629

[3] K. Greshake et al., “Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” arXiv:2302.12173, 2023. https://arxiv.org/abs/2302.12173

[4] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” arXiv:2307.03172, 2023. https://arxiv.org/abs/2307.03172

[5] S. Es, J. James, L. Espinosa Anke and S. Schockaert, “RAGAs: Automated Evaluation of Retrieval Augmented Generation,” EACL System Demonstrations, pp. 150-158, 2024. https://aclanthology.org/2024.eacl-demo.16/

[6] pgvector maintainers, “pgvector documentation,” accessed 8 September 2026. https://github.com/pgvector/pgvector

[7] Ollama, “Generate a chat message,” API documentation, accessed 8 September 2026. https://docs.ollama.com/api/chat

Appendix A. Evidence manifest

Identifier	File in evidence/	Role
A1	production-stress.json	Four API load stages
A2	ai-stress.json	Short concurrent AI-task run
A3	production-e2e.json	Ten named live workflow checks
A4	eval-1788679222.json	Earlier keyword-coverage pilot
A5	recovery-check.json; fault-recovery.json	Database/count and restart behavior
A6	validation-tests.log; race-tests.log	Separate build/unit/integration/race records
A7	model-pilot.json	Full new requests, outputs, timing and identity
A8	model-pilot-review.json	Transparent post-hoc labels and summaries
A9	chunk-sensitivity.json	Actual chunker count experiments
A10	provenance.json	Source/evidence hashes and source-state caveats

Appendix B. Minimal replication protocol

Use the existing local models and execute source/run_model_pilot.py. Its requests use two temperatures, two seeds, three fixed cases and a per-model warmup, saving all outputs as JSON. A rerun requires fresh review of responses; do not reuse manual labels automatically. Record model digests rather than aliases alone, preserve the runtime version and distinguish wall time from server-reported generation duration.

For an end-to-end comparison, create an isolated workspace per variant, ingest an identical versioned corpus with one fixed embedder, prevent prior generated memories from contaminating later trials and run the same questions through the authenticated task API. A model-only generation pilot is insufficient to measure retrieval or authorization effects. For an embedding comparison, rebuild the corpus embeddings and recalibrate on a development split before evaluating the held-out questions.